Qwen

Architecting the Constellation-Contracts Physics Layer: A Schema-First Approach to Integrated AI System Governance

Foundational Principles of the Constellation-Contracts Framework

The development of a robust and scalable governance architecture for complex agentic AI systems necessitates a foundational shift away from ad-hoc, post-hoc oversight mechanisms toward a more integrated, machine-enforceable model [16] [21]. The Constellation-Contracts thesis, as articulated through the initial three specified modules—AI-Chat Template & Redaction, Telemetry & Entertainment Metrics, and History-Aware Content Selector—proposes the creation of an immutable "physics layer" [55]. This layer is not merely a conceptual abstraction but a concrete technical implementation built upon three core principles: an immutable schema spine, explicit cross-referencing for integration, and a standardized yet extensible interface layer. These principles collectively aim to establish a system where policies are encoded as verifiable contracts, enabling continuous integration (CI) enforcement, AI-assisted authoring, and unified telemetry convergence. The success of this framework hinges on the rigorous application of these foundational pillars before any significant integrations are attempted, ensuring that the system's logical and data integrity is preserved.

The first principle, the **Immutable Schema Spine**, establishes a set of canonical, versioned JSON schemas as the single source of truth for the system's logic and data structures [40] [42]. This concept treats the core data objects not as mutable configurations but as immutable artifacts stored within a dedicated repository, `HorrorPlace-Constellation-Contracts`. This approach provides a stable foundation upon which the entire system can be built and audited. The evidence for this principle is clearly demonstrated in the initial modules. Module 1 (AI-Chat Template & Redaction) specifies the need for a `Canonical AI-Chat Template Schema Object` and a `Redaction Language & Profile Schema Object`. These are intended for direct implementation and subsequent use in CI workflows, such as leak-checks, to ensure that no template violates its predefined safety contract . Similarly, Module 2 (Telemetry & Entertainment Metrics) defines a `entertainmentmetricsv1` schema with explicit

`[0,1]` bands and calculation hints, transforming abstract metrics into enforceable data contracts that can be validated automatically . Module 3 (History-Aware Content Selector) introduces a `Selector DSL Schema & Pattern Library Object` that formally maps narrative invariants to specific content patterns, creating a structured and governable decision-making process . In each case, the transition from a conceptual "gap" to a tangible, implementable artifact marks the stabilization of a piece of the system's physics layer. These schemas serve as the atomic units of the system's logic, providing a predictable and consistent basis for all downstream operations.

The second principle, **Explicit Cross-Referencing as Integration**, fundamentally rethinks how disparate modules interact. Instead of relying on external configuration files or loosely coupled APIs, integration is baked directly into the structure of the data itself. Each module's schema contains fields designed to hold identifiers that point to objects created by other modules. This creates a self-documenting graph of interconnected data, where every event carries the context of its origin and dependencies. The user's directive to include fields that reference shared objects like `consentTier`, `routerStateId`, and `selectorPatternId` is the practical manifestation of this principle . The conversation also identifies a core set of common join keys—`sessionId`, `regionId`, `seedId`, and `descriptorId`—that will be present across all modules to enable session-level correlation . The mechanism is straightforward: when a module generates a telemetry event, it embeds the relevant ID from another module's object within that event's payload. For instance, a `GovernanceEvent` that deprecates a content pattern would include the `patternId` of the affected object, while a `SelectorDecisionEvent` would contain the `selectorPatternId` that was chosen . This embedded referencing allows for powerful causal analysis; any event in the system's NDJSON log stream can be traced backward through these links to understand the full chain of decisions that led to it. This approach is analogous to the concept of a digital twin, where every action and state change is meticulously recorded with its contextual metadata, providing diagnostic value for policy evaluation and system behavior analysis [4] [7].

The third principle, **Standardized but Extensible Interfaces**, ensures that while each module possesses unique domain logic, they interact through a consistent and predictable surface. This prevents the emergence of a tangled web of custom, undocumented interfaces and enables tools like test harnesses and AI-authoring assistants to reason about the system as a whole. The research specifies a clear convention for the shared Lua API surface, centered around the naming pattern `H.Module.verbNoun()` (e.g., `H.Templates.select`, `H.Metrics.recordEvent`) . This structure provides a discoverable namespace that logically groups functionality. Furthermore, it mandates a minimal common data contract for function returns and parameters. Every interaction must accept an explicit `sessionContext` or `sessionId` and return a structured `{ok,`

`data, error}` object . This standardizes state passing and error handling, making code more robust and easier to debug. Finally, a common telemetry hook, such as `H.Telemetry.emit(event)`, is required, ensuring that all events emitted by modules follow a consistent format (NDJSON) and adhere to the established join-key conventions . This balanced approach allows a `Budget` module, for example, to have its own rich set of functions and data shapes, such as tracking `budgetState`, while guaranteeing that any other part of the system can reliably call its functions and understand its outputs. It mirrors the best practices of microservices architecture, where services communicate over well-defined, typed APIs, promoting loose coupling and independent deployability [37] [38].

These three principles—the immutable schema spine, explicit cross-referencing, and standardized interfaces—are not merely technical guidelines but the architectural embodiment of the Constellation-Contracts thesis. They collectively create a system that is transparent, auditable, and capable of dynamic adaptation based on verifiable data. By finalizing the core schemas for the remaining seven modules and embedding these integration points from the outset, the project moves closer to realizing this vision of a truly governed, intelligent system. This schema-first approach is the critical next step in building the stable, foundational layer that will support AI-authoring contracts, enforce compliance at scale, and provide a unified view of system performance and behavior.

Schema Definitions for Entry Point and User-Centric Modules

The foundational pillars of the Constellation-Contracts framework—the immutable schema spine, explicit cross-referencing, and standardized interfaces—must now be applied to define the core JSON schemas for the remaining seven modules. This section focuses on the first two modules in this set: the Age-Gated Tier Router and the Consent Profile & Comfort module. These modules represent the system's entry point and its primary interface with the user's identity and preferences. Their schemas are crucial because they determine the baseline ruleset for every subsequent interaction, effectively setting the stage for the entire user experience. The definitions provided below are minimal, stable, and explicitly designed to integrate with the existing three modules and the established set of cross-referencing IDs.

Module 4: Age-Gated Tier Router

This module serves as the system's initial gatekeeper, translating a user's demographic information and consent profile into a specific safety tier. This tier dictates the permissible content, intensity levels, and policy constraints for the duration of the session. Its primary output is the `consentTier`, which acts as a key that unlocks the appropriate configurations in downstream modules like the AI-Chat Template & Redaction and the History-Aware Content Selector. The core schema for this module, tentatively named `tierRouterContractv1.json`, formalizes this decision-making process into a machine-readable contract.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/tierRouterContractv1.json`. [40]
`type`	string	Must be `"object"`. [121]
`title`	string	Human-readable title: `"Age-Gated Tier Router Contract"`.
`description`	string	Detailed explanation of the schema's purpose: "Defines the logic for assigning a user to a consent tier based on their attributes."
`required`	array	List of required properties: `["userId", "age", "userConsentProfileId", "effectiveAt"]`. [40]
`properties`	object	Contains the definitions for each field in the schema.
`routerStateId`	string	A unique identifier for this specific router decision instance, generated by the router module. It follows the pattern `^(rtr|router)-[a-z0-9]{8}$`. This ID links back to the router's state and is passed to other modules for auditability. [11]
`userId`	string	The unique identifier for the user being processed.
`age`	integer	The user's age, must be a non-negative integer.
`userConsentProfileId`	string	A reference to the `profileId` of the `consentProfilev1.json` object used for this decision. This embeds the link to the user's broader consent settings. [120]
`consentTier`	string	The resulting safety tier assigned to the user (e.g., `'Tier 0'`, `'Tier 1'`, `'Tier 2'`). This field is the primary output of the router and must correspond to a tier defined in the `template` schema of Module 1. The enum values would be defined based on the project's specific tiers. [115]
`decisionTimestamp`	string	Timestamp of when the routing decision was made, formatted as a date-time string. [119]

The integration points for the `tierRouterContractv1.json` schema are fundamental to its role. It **references `userConsentProfileId`**, creating a direct link between a routing decision and the user's static consent preferences managed by the Consent Profile module. This allows for auditing and ensures that the router is operating on the most current consent information. Its most critical integration is its **output of `consentTier`**. This string becomes a primary key used by numerous downstream modules. The AI-Chat Template module will use it to select the correct redaction profile, the Content Boundary module will use it to apply the corresponding policy rules, and the Intensity Budget

module may use it to set initial budget limits. Finally, the generation of a unique `routerStateId` provides a crucial tracing mechanism, allowing a `sessionMetricsEnvelopeId` from Module 2 to be linked back to the precise moment and conditions under which the user was assigned to a particular tier.

Module 7: Consent Profile & Comfort

This module manages the user's explicit consent settings and dynamically tracks their comfort level throughout a session. It acts as the central repository for both static preferences (e.g., "no graphic violence") and dynamic indicators derived from behavioral signals (e.g., UEC dropping below a threshold). Its schema, `consentProfilev1.json`, must accommodate both these facets of user control. This module's output directly influences the Tier Router and provides real-time feedback to the Content Boundary and potentially the Intensity Budget.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/consentProfilev1.json`. [40]
`type`	string	Must be `"object"`. [121]
`title`	string	Human-readable title: `"User Consent Profile"`.
`description`	string	Detailed explanation of the schema's purpose: "Manages a user's consent settings and comfort level indicators."
`required`	array	List of required properties: `["profileId", "userId"]`. [40]
`properties`	object	Contains the definitions for each field in the schema.
`profileId`	string	The unique identifier for this consent profile object. This serves as the primary key referenced by the Tier Router's `userConsentProfileId` field. [122]
`userId`	string	The unique identifier for the user associated with this profile.
`staticPreferences`	object	An object containing the user's fixed consent choices.
`explicitContentAllowed`	boolean	A flag indicating if the user has opted-in to explicit content.
`noViolenceAllowed`	boolean	A flag indicating if the user has selected a preference for no violence.
`comfortIndicators`	object	An object containing dynamically calculated comfort metrics.
`comfortScore`	number	A floating-point score between 0 and 1 representing the user's current comfort level. This value would be updated based on signals from the Telemetry module (Module 2). [133]
`lastComfortUpdate`	string	Timestamp of the last time the comfort score was updated, formatted as a date-time string.
`lastUpdated`	string	Timestamp of when this consent profile object was last modified, formatted as a date-time string.

The integration points for the `consentProfilev1.json` schema are equally vital. Its `profileId` is the primary key that allows the **Age-Gated Tier Router to look up the user's consent settings** when a new session begins. This establishes the foundational link between the two modules. The `staticPreferences` object directly **informs the Content Boundary & Policy Engine**. If `noViolenceAllowed` is true, the engine can apply stricter filters regardless of the current session's `consentTier`. This provides a hard override for user preferences. The most dynamic integration point is the `comfortIndicators` object. The `comfortScore` is a value that would be continuously updated by logic consuming telemetry from the Telemetry & Entertainment Metrics module. For example, if a user's UEC drops significantly or CDL rises above a certain threshold, this score could be decremented. Conversely, positive engagement signals could increment it. This `comfortScore` could then be used to trigger actions elsewhere in the system, such as prompting the Intensity Budget to reduce its rate of consumption or signaling the Editor Copilot to suggest less intense narrative patterns. The `profileId` also serves as a general-purpose join key, allowing any governance event related to consent (e.g., a user changing their settings) to be linked back to this specific user object via the `targetDescriptorId` field in the `governanceEventv1.json` schema.

Schema Definitions for Session-State and Content Moderation Modules

Building upon the foundational schemas for user entry and consent management, this section details the core JSON schemas for modules that govern the internal state and safety of a user's session. The Intensity Budget, Content Boundary & Policy Engine, and History-Aware Content Selector form the operational heart of the system, responsible for maintaining balance, enforcing safety rules, and driving the narrative. Their schemas must be precise, capturing not only the state of their respective domains but also the critical links back to the initial user context and to each other. By defining these schemas with explicit cross-references, we create a cohesive and analyzable record of every session's journey, enabling deep diagnostics and adaptive learning.

Module 5: Intensity Budget

This module is tasked with managing the cumulative intensity of a user's experience, acting as a guardrail against desensitization or excessive distress. It operates on the principle of an "Intensity Unit" budget that is consumed by selector decisions and content

generation. The schema for this module, `intensityBudgetEnvelopev1.json`, is designed to capture the state of this budget and the events that modify it, providing a clear audit trail of intensity exposure.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/intensityBudgetEnvelopev1.json`.
`type`	string	Must be `"object"`.
`title`	string	Human-readable title: `"Intensity Budget Envelope"`.
`description`	string	Detailed explanation of the schema's purpose: "An envelope containing the current state of the intensity budget and related events."
`required`	array	List of required properties: `["budgetId", "sessionId", "currentIntensity", "maxIntensity"]`.
`properties`	object	Contains the definitions for each field in the schema.
`budgetId`	string	The unique identifier for this specific budget instance.
`sessionId`	string	The unique identifier for the session this budget belongs to. This is a critical join key for correlating budget events with telemetry from other modules.
`currentIntensity`	number	The amount of intensity units currently consumed, with a minimum value of 0.
`maxIntensity`	number	The total capacity of the intensity budget, with a minimum value of 0.
`isOverBudget`	boolean	A flag indicating whether the current intensity has exceeded the maximum budget.
`events`	object	An array of objects representing the history of budget modifications.
`eventId`	string	A unique identifier for the event.
`eventType`	string	The type of event, constrained to an enumeration: `["consumption", "refill", "reset"]`.
`amount`	number	The quantity of intensity units added or removed, with a minimum value of 0.
`sourcePatternId`	string	A reference to the `selectorPatternId` from the History-Aware Content Selector (Module 3) that triggered a consumption event. This links intensity cost directly to narrative choices.

The integration points for the `intensityBudgetEnvelopev1.json` schema are essential for its function as a dynamic control mechanism. The inclusion of `sessionId` makes it readily joinable with the `sessionMetricsEnvelopeId` from the Telemetry & Entertainment Metrics module, allowing for the analysis of how intensity consumption correlates with entertainment metrics like UEC and ARR. The most significant integration is the **reference to `selectorPatternId`** within the `events` array. When the selector chooses a pattern that has an associated intensity cost, it logs this event, embedding the pattern's ID. This creates a direct causal link, enabling analysts to ask questions like, "Which specific narrative patterns are driving the highest intensity consumption?" or "Does high intensity consumption correlate with a drop in UEC?" The `isOverBudget` boolean flag serves as a real-time signal to other modules. If this flag is true, it could

trigger the Content Boundary & Policy Engine to become more restrictive or prompt the Tier Router to consider downgrading the user's tier, thus closing a feedback loop for session management.

Module 3 (Revisited for Integration): History-Aware Content Selector

While the selector's core logic is defined by its DSL, its output events must conform to a standardized logging schema that integrates with the broader system. The `Selector Decision Event & Logging Object` is the mechanism for this integration, capturing the outcome of the selector's choice along with all relevant contextual data . This event is logged by every invocation of `H.Selector.choosePattern`.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/selectorDecisionEventv1.json`.
`type`	string	Must be `"object"`.
`title`	string	Human-readable title: `"Selector Decision Event"`.
`description`	string	Detailed explanation of the schema's purpose: "Opaque telemetry event capturing a selector's decision and its context."
`required`	array	List of required properties: `["eventId", "decisionTimestamp", "regionId", "chosenPatternId", "invariantsSnapshot"]`.
`properties`	object	Contains the definitions for each field in the schema.
`eventId`	string	A unique identifier for this decision event.
`decisionTimestamp`	string	Timestamp of when the selection was made, formatted as a date-time string.
`sessionId`	string	Join key for correlating this decision with other session-wide events.
`regionId`	string	Join key for identifying the specific region in the game world where the decision was made.
`chosenPatternId`	string	The unique identifier of the narrative pattern that was selected. This links the decision to the pattern library.
`invariantsSnapshot`	object	A snapshot of the current invariant values at the time of the decision. This includes fields like CIC, MDI, AOS, DET, etc.
`userIntentTags`	array	Tags representing the user's expressed intent for this turn.
`resolutionStrategy`	string	A description of how any conflicts were resolved (e.g., "denied_conflict", "softened_intent", "rerouted_to_region_X").
`metadata`	object	Optional field for module-specific data, such as the `explicitnessBand` selected.

The `selectorDecisionEventv1.json` schema is a cornerstone of the telemetry fabric. It **references `sessionId` and `regionId`**, making it a node in the network of session-level events. Its direct reference to `chosenPatternId` links the decision back to the `Selector DSL Schema & Pattern Library Object` from Module 3,

allowing for detailed analysis of pattern popularity and effectiveness. The inclusion of a full `invariantsSnapshot` provides the raw data needed for the Telemetry & Entertainment Metrics module to calculate band violations and update its segment trackers. Most importantly, the `resolutionStrategy` field captures the outcome of the conflict resolution policy, providing invaluable data for tuning the `Conflict Resolution Policy & Routing Object` and improving the selector's ability to handle user intent mismatches.

Module 6: Content Boundary & Policy Engine

This module is the runtime guardian of safety and ethics, performing real-time checks on generated content. Its output is a verdict that informs the AI chat, the editor, and potentially the session's budget. The schema for its judgment events, `contentBoundaryJudgmentv1.json`, must be precise, capturing the content being judged, the verdict, and the reasoning behind it.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/contentBoundaryJudgmentv1.json`.
`type`	string	Must be `"object"`.
`title`	string	Human-readable title: `"Content Boundary Judgment"`.
`description`	string	Detailed explanation of the schema's purpose: "The result of a real-time content safety check."
`required`	array	List of required properties: `["judgmentId", "contentHash", "verdict", "violations"]`.
`properties`	object	Contains the definitions for each field in the schema.
`judgmentId`	string	A unique identifier for this judgment event.
`contentHash`	string	A SHA-256 hash of the content being judged. This ensures identical content is not re-evaluated unnecessarily. It must match the regex pattern `^[a-f0-9]{64}$`.
`verdict`	string	The outcome of the check, constrained to an enumeration: `["ALLOWED", "REDIRECT", "REFUSE", "SOFTEN"]`.
`violations`	array	A list of detected harm categories and their severity levels.
`harmCategory`	string	The type of violation detected (e.g., "Graphic Violence," "Hate Speech"). This can be drawn from standards like Azure AI Content Safety's harm categories. [43]
`severityLevel`	integer	An integer from 1 to 5 indicating the severity of the violation.
`policyAppliedId`	string	A reference to the unique ID of the specific policy rule that was triggered. This provides a direct audit trail to the governance ledger.
`templateId`	string	A reference to the `templateId` from the AI-Chat Template & Redaction module (Module 1) that was used to generate the content. This links the verdict back to the original contractual agreement.

The integration points for the `contentBoundaryJudgmentv1.json` schema are critical for a closed-loop governance system. The **reference to `templateId`** is paramount. If a piece of content is refused, knowing which template it came from allows the Editor Copilot to suggest corrections that are compliant with that specific contract. The `policyAppliedId` provides a direct link to the `DeadLedger/ALN Governance` module, creating an unbroken chain of custody from policy proposal to runtime enforcement. The `verdict` field (`ALLOWED`, `REDIRECT`, `REFUSE`, `SOFTEN`) is the primary communication channel to other modules. An `ALLOWED` verdict proceeds normally. A `REFUSE` or `REDIRECT` verdict would cause the AI Chat to halt or reroute its narrative, while a `SOFTEN` verdict would trigger a call to the `redact/soften/imply/refuse` policy router defined in Module 1, initiating a redaction process. This schema transforms the Content Boundary from a simple filter into a fully instrumented component whose actions are logged, auditable, and directly influence the flow of the entire system.

Schema Definitions for Authoring, Procedural Generation, and Governance Modules

Beyond governing the live user experience, the Constellation-Contracts framework extends to the processes of content creation, world generation, and system evolution. This requires specialized modules with their own unique responsibilities and corresponding schemas. The Editor Copilot facilitates human-authored content, the PCG Seed Generator initializes the game world, and the DeadLedger/ALN Governance provides an immutable record of all system-changing decisions. The schemas for these modules are designed to capture the essence of their domain while adhering to the overarching principles of the framework, particularly the use of standardized join keys to maintain a cohesive data fabric.

Module 8: Editor Copilot

The Editor Copilot is an AI-assisted tool designed to help human editors create, review, and manage content governed by the system's contracts. Its primary function is to reduce cognitive load and prevent errors by automating checks, generating suggestions, and providing context-aware assistance. The core of its operation is a detailed, auditable log of its interactions. The schema for these logs, `copilotActionLogv1.json`, captures every suggestion, check, and change made during an editing session.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/copilotActionLogv1.json`.
`type`	string	Must be `"object"`.
`title`	string	Human-readable title: `"Editor Copilot Action Log"`.
`description`	string	Detailed explanation of the schema's purpose: "Logs every action taken by the Editor Copilot during content authoring."
`required`	array	List of required properties: `["logId", "authoringSessionId", "actorId", "actionType", "targetObjectId", "details"]`.
`properties`	object	Contains the definitions for each field in the schema.
`logId`	string	A unique identifier for this specific log entry.
`authoringSessionId`	string	A join key that groups all actions performed during a single editing workflow or pull request.
`actorId`	string	The unique identifier of the human editor who invoked the copilot's action.
`actionType`	string	The nature of the action, constrained to an enumeration: `["suggestion", "check", "generation", "refactoring"]`.
`targetObjectType`	string	The type of object being edited (e.g., `"Template"`, `"Pattern"`, `"PolicyRule"`).
`targetObjectId`	string	A reference to the unique ID of the object being acted upon (e.g., a `templateId` or `selectorPatternId`). This creates a direct link to the governed object.
`details`	object	An object containing specifics about the action.
`suggestionText`	string	The text of a suggestion provided by the copilot.
`checkResults`	object	The results of automated checks performed by the copilot (e.g., schema validation, leak-test pass/fail).
`generatedCode`	string	The code or content generated by the copilot.
`timestamp`	string	Timestamp of when the action was logged, formatted as a date-time string.

The integration points for the `copilotActionLogv1.json` schema are what make it a powerful tool for quality assurance and knowledge transfer. The **reference to `targetObjectId`** is the most critical link, connecting the Copilot's action directly to the content it was applied to. This allows for precise auditing of changes and understanding the rationale behind them. The schema leverages the **standardized Lua API surface** by design; the Copilot itself would be implemented as a script that calls `H.Templates`, `H.Selector`, and other module APIs to perform its checks and generate suggestions, thereby adhering to the established interaction patterns . Furthermore, these action log events become part of the overall NDJSON telemetry stream. By joining them with events from the `Selector` and `Metrics` modules, one can analyze the impact of the Editor Copilot—for example, by measuring whether sessions involving content authored with Copilot assistance exhibit higher compliance rates or better entertainment metrics.

Module 10: PCG Seed Generator

This module is responsible for generating the procedural seeds that initialize the game world and influence the emergent narrative. These seeds determine aspects like world style, initial layout, and difficulty, providing a unique starting point for each playthrough. The schema for its output, `pcgSeedGenerationv1.json`, records the parameters used and the resulting seed value, creating a reproducible and analyzable record of world generation.

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/pcgSeedGenerationv1.json`.
`type`	string	Must be `"object"`.
`title`	string	Human-readable title: `"PCG Seed Generation Record"`.
`description`	string	Detailed explanation of the schema's purpose: "Records the parameters and outcome of a procedural content generation seed."
`required`	array	List of required properties: `["seedId", "generationRequestId", "generatorVersion", "parameters", "seedValue"]`.
`properties`	object	Contains the definitions for each field in the schema.
`seedId`	string	A unique identifier for this generated seed. This is a crucial join key, linking the initial world state to all subsequent events.
`generationRequestId`	string	A unique identifier for the request that triggered the seed generation.
`generatorVersion`	string	The version of the PCG algorithm used to generate the seed.
`parameters`	object	An object containing the input parameters for the generator.
`worldStyle`	string	The desired aesthetic or thematic style of the world (e.g., "Lovecraftian", "Slasher", "Found Footage").
`difficultyLevel`	string	The initial difficulty setting (e.g., "Normal", "Nightmare").
`initialRegionId`	string	The ID of the first region the player spawns in.
`seedValue`	integer	The actual integer value generated by the PCG algorithm.
`metadata`	object	An optional field for storing additional statistics or properties about the generated seed, such as region layout complexity or resource distribution.

The `pcgSeedGenerationv1.json` schema plays a pivotal role in enabling cohort-based analysis. The `seedId` is the linchpin of this capability; it serves as a master join key that connects the initial world setup to every subsequent event logged in the system, including selector choices, content boundary judgments, and final entertainment metrics. This allows researchers to answer critical questions, such as, "How do sessions generated with a 'Lovecraftian' style compare to those with a 'Slasher' style in terms of UEC and CDL?" or "Does a higher initial difficulty level lead to faster depletion of the Intensity Budget?" The `parameters` object provides valuable context to the **History-Aware Content Selector (Module 3)**. Knowing the `worldStyle` and `difficultyLevel` at

the start of the session allows the selector to adapt its strategy, perhaps choosing different patterns or adjusting its sensitivity to user intent. For example, in a "Lovecraftian" world, the selector might prioritize building dread over sudden gore, even if the user requests it. This schema transforms the PCG process from a black box into a transparent, data-driven component of the larger system.

Module 9: DeadLedger/ALN Governance

This module serves as the ultimate source of truth for all system governance decisions. It is an append-only ledger that records proposals, votes, locks, unlocks, and deprecation notices. Its schema, `governanceEventv1.json`, must be flexible enough to represent a wide variety of administrative actions while providing hooks for linking these events to the specific objects they affect. This ledger is the foundation for verifiable compliance and institutional memory [79].

Field Name	Type	Constraints / Description
`$id`	string	Unique URL identifier for the schema: `https://horrorplace.org/schemas/governanceEventv1.json`.
`type`	string	Must be `"object"`.
`title`	string	Human-readable title: `"Governance Event"`.
`description`	string	Detailed explanation of the schema's purpose: "A standardized event representing a proposal, vote, or administrative action."
`required`	array	List of required properties: `["eventId", "eventType", "proposerId", "timestamp"]`.
`properties`	object	Contains the definitions for each field in the schema.
`eventId`	string	A unique identifier for this governance event.
`eventType`	string	The category of the event, constrained to an enumeration: `["proposal", "vote", "lock", "unlock", "deprecation"]`.
`proposerId`	string	The unique identifier of the user or agent that initiated the event.
`targetDescriptorId`	string	A general-purpose join key to link the event to the object it concerns (e.g., the `templateId` or `patternId` being deprecated).
`targetRulesetId`	string	A general-purpose join key to link the event to a specific ruleset or policy being modified.
`proposalDetails`	object	Details specific to a "proposal" event, such as the new policy text or the rationale for the change.
`voteResults`	object	The outcome of a voting process, including voter IDs and their choices.
`timestamp`	string	Timestamp of when the event occurred, formatted as a date-time string.

The `governanceEventv1.json` schema is the bedrock of the system's procedural governance. Its primary function is to provide an immutable and auditable record of all changes. The **general-purpose join keys `targetDescriptorId` and**

`targetRulesetId` are what give it power. When a policy is changed, its ID is placed here, creating a direct, traceable link between the governance decision and the object it governs. This allows for powerful diagnostics, such as reconstructing why a particular template was modified based on past votes. This module triggers actions in others: a successful `vote` event could trigger a webhook to the **Editor Copilot** to begin drafting the implementation of a newly approved template. A `deprecation` event would signal to other modules to stop using the specified object. Finally, the ledger provides the historical context that informs the long-term tuning of the **Selector** and **Intensity Budget**. For instance, if a certain type of pattern was consistently voted down or deprecated, the selector's pattern library could be adjusted to deprioritize similar motifs in the future. This schema operationalizes the governance aspect of the framework, moving it from a theoretical concept to a practical, data-driven process.

Synthesized Analysis of Cross-Module Integration and Operational Implications

The systematic definition of core JSON schemas for the seven remaining governance modules, grounded in the foundational principles of the Constellation-Contracts thesis, yields a comprehensive and coherent system architecture. This schema-first approach, which prioritizes stabilizing the data contracts before layering on integrations, is not merely an architectural preference but a strategic necessity for achieving verifiable compliance, enabling sophisticated telemetry analysis, and fostering a culture of AI-assisted development. The synthesis of these individual schemas reveals a tightly integrated ecosystem where data flows are explicit, traceable, and governed by machine-readable contracts. The operational implications of this design are profound, promising enhanced transparency, improved system reliability, and the ability to evolve and adapt based on empirical evidence.

The most significant implication of this schema-centric design is the creation of a **unified and analyzable data fabric**. By baking in a common set of join keys—including `sessionId`, `regionId`, `seedId`, and `descriptorId`—across all modules, the system ensures that disparate event streams, though originating from different domains (e.g., content generation, user metrics, governance), can be joined and correlated . This transforms the collection of NDJSON logs from a series of isolated events into a rich, multi-dimensional dataset. For example, an analyst can now construct a complete causal chain: starting from a `governanceEventv1.json` that deprecates a specific `templateId`, they can trace the impact of this decision by filtering

`copilotActionLogv1.json` events related to that template, examining `selectorDecisionEventv1.json` events to see how often it was chosen before the deprecation, and finally analyzing the `sessionMetricsEnvelopev1.json` entries from sessions that included it to quantify its effect on UEC and ARR. This level of deep, retrospective analysis was previously infeasible in systems with siloed data and opaque integrations. It aligns with the goals of a digital twin, providing a diagnostic tool for policy evaluation and system assessment [4] [7].

Furthermore, this framework enables the construction of **closed-loop feedback systems** that drive continuous improvement. The integration points are not just for passive observation; they are active components in a cycle of monitoring, learning, and adapting. The `comfortScore` from the `consentProfilev1.json` schema, influenced by telemetry from the `entertainmentmetricsv1.json` schema, can trigger adjustments in the `Intensity Budget`'s refill rate. A `ContentBoundaryJudgmentv1.json` event that results in a `REFUSE` verdict provides immediate feedback to the `Editor Copilot`, guiding human authors toward more compliant creations. The `ConflictResolutionPolicy` in the `History-Aware Content Selector` can be tuned based on telemetry showing which conflict resolutions lead to negative user sentiment. This dynamic interplay, orchestrated by the explicit cross-referencing of schemas, moves the system beyond static, pre-defined rulesets toward a more adaptive and responsive mode of governance. It embodies the principles of trustworthy orchestration and control-theoretic architectures, where system behavior is guided by a combination of ethics, real-time data, and governance [12] [128].

However, the successful implementation of this framework also reveals several **critical operational considerations and potential gaps**. First, the scalability of the `DeadLedger/ALN Governance` module presents a challenge. While the `governanceEventv1.json` schema is intentionally simple to ensure immutability, a high volume of governance events could strain storage and querying capabilities. It may be necessary to implement a separate, optimized database or indexing strategy for this ledger, distinguishing its use case (immutable audit log) from that of the operational telemetry streams. Second, the mechanism for updating dynamic fields, such as the `comfortIndicators` in the `consentProfilev1.json` schema, requires further specification. The logic for calculating the `comfortScore`—the weight of different signals, the rate of decay, and the thresholds for adjustment—is a critical piece of business logic that needs to be codified. Third, the coordination of decisions across multiple modules introduces a need for a higher-level orchestrator. The logic for resolving a situation where a user's intent conflicts with the `Intensity Budget` being over its limit, the `Content Boundary` rejecting the content, and a `GovernanceEvent` locking the requested feature, is not trivial. This requires an aggregation or orchestrator

module not yet detailed in the framework, which would consume events from multiple sources and make a final, composite decision, likely embodying a form of runtime governance for the system itself [93] [94].

In conclusion, the definition of these seven core JSON schemas represents a decisive step toward realizing the Constellation-Contracts thesis. By establishing a stable, immutable schema spine and weaving explicit cross-references into the fabric of the data, the project has laid the groundwork for a system that is transparent, auditable, and capable of self-correction. The standardized yet extensible API surface ensures that while each module retains its domain-specific intelligence, they can collaborate through a consistent and predictable interface. The operational implications are substantial, offering the promise of unprecedented insight into system dynamics and the ability to build truly adaptive and responsible AI systems. While challenges related to scalability, dynamic logic, and inter-module coordination remain, the schema-first approach provides a robust and principled path forward for addressing them. This framework is not just a technical solution; it is a blueprint for a new paradigm of AI governance, one where rules are written in code, enforced by machines, and refined by data.

Reference

1. (PDF) Cryptocurrency Constellations Across the Three-Dimensional ... https://www.researchgate.net/publication/344565111_Cryptocurrency_Constellations_Across_the_Three-Dimensional_Space_Governance_Decentralization_Security_and_Scalability
2. [PDF] Network Digital Twin for 6G and Beyond - arXiv https://arxiv.org/pdf/2506.01609
3. [PDF] UC Santa Cruz Electronic Theses and Dissertations - eScholarship.org https://escholarship.org/content/qt8cx0j148/qt8cx0j148.pdf
4. LLM-Augmented Digital Twin for Policy Evaluation in Short-Video ... https://arxiv.org/html/2603.11333v1
5. [PDF] MI9: An Integrated Runtime Governance Framework for Agentic AI https://openreview.net/pdf?id=TseVPnC26W
6. A Review of TRiSM Frameworks in Artificial Intelligence Systems ... https://onlinelibrary.wiley.com/doi/10.1111/exsy.70213

7. (PDF) LLM-Augmented Digital Twin for Policy Evaluation in Short ... https://www.researchgate.net/publication/401909503_LLM-Augmented_Digital_Twin_for_Policy_Evaluation_in_Short-Video_Platforms
8. Fabric March 2026 Feature Summary | Microsoft Fabric Blog https://blog.fabric.microsoft.com/en-us/blog/fabric-march-2026-feature-summary?ft=Ai:category
9. Generative AI-based approach for player behavior analysis and gray ... https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1730018/full
10. Ali Šifrar's Post - LinkedIn https://www.linkedin.com/posts/alizzzzz_your-6-month-data-strategy-produced-a-slide-activity-7389279644006383617-YQVX
11. Parth P. - LinkedIn https://www.linkedin.com/in/parth-panchmatia
12. [PDF] Trustworthy Orchestration Artificial Intelligence by the Ten Criteria ... https://arxiv.org/pdf/2512.10304
13. A Framework for Responsible AI Systems: Building Societal Trust ... https://arxiv.org/html/2503.04739v2
14. Defining AI Models and AI Systems: A Framework to Resolve the ... https://arxiv.org/html/2603.10023
15. A Scalable Framework for Multi-Policy AI Compliance Evaluation https://arxiv.org/html/2601.11702v1
16. The Controllability Trap: A Governance Framework for Military AI ... https://arxiv.org/html/2603.03515v1
17. Governance-as-a-Service: A Multi-Agent Framework for AI System ... https://arxiv.org/html/2508.18765v2
18. Executable Governance for AI: Translating Policies into Rules Using ... https://arxiv.org/html/2512.04408v1
19. A Comprehensive Survey on AI Governance - arXiv https://arxiv.org/html/2508.08789v2
20. [PDF] A Five-Layer Framework for AI Governance: Integrating Regulation ... https://arxiv.org/pdf/2509.11332
21. Cryptographic Runtime Governance for Autonomous AI Systems https://arxiv.org/html/2603.16938v1
22. A Security Governance Specification for AI Agent Tool Access - arXiv https://arxiv.org/html/2602.20196v1
23. How to Write a Good Spec for AI Agents – O'Reilly https://www.oreilly.com/radar/how-to-write-a-good-spec-for-ai-agents/

24. Emails not sent due to wrong email address syntax - SAP Help Portal https://help.sap.com/docs/SAP_EMARSYS/f8e2fafeea804018a954a8857d9dfff3/fdecf5bb74c11014b59cc2760245a63e.html?url_id=text-de-newscenter:CRM-DX26-STR-455646_01
25. Self-Sovereign Identity: A Systematic Review, Mapping and Taxonomy https://pmc.ncbi.nlm.nih.gov/articles/PMC9371034/
26. Microsoft 365 Roadmap | Microsoft 365 https://www.microsoft.com/microsoft-365/roadmap?featureid=396769
27. About GitHub Copilot cloud agent https://docs.github.com/copilot/concepts/agents/coding-agent/about-coding-agent
28. Fabric March 2026 Feature Summary | Microsoft Fabric blog https://blog.fabric.microsoft.com/sr-latn-rs/blog/fabric-march-2026-feature-summary?ft=Noelle%20Li:author
29. RAG based on SharePoint data: A flawed approach - LinkedIn https://www.linkedin.com/posts/brianfending_sharepoint-at-25-global-enterprise-knowledge-activity-7434968572331786240-YKlp
30. MATLAB 发行说明 https://ww2.mathworks.cn/help/matlab/release-notes.html
31. TowardsDataScience-博客中文翻译-2020-二十五- - 绝不原创的飞龙 https://www.cnblogs.com/apachecn/p/18463796
32. Natural Language Processing and Applications https://link.springer.com/content/pdf/10.1007/978-981-97-9739-4.pdf
33. Is PowerShell ready to replace my Cygwin shell on Windows? [closed] https://stackoverflow.com/questions/573623/is-powershell-ready-to-replace-my-cygwin-shell-on-windows
34. Antenna Array Optimization For Side Lobe Level A Brief Review https://www.scribd.com/document/684255290/Antenna-Array-Optimization-for-Side-Lobe-Level-a-Brief-Review
35. PCG开发指南| 虚幻引擎5.7 文档| Epic Developer Community https://dev.epicgames.com/documentation/zh-cn/unreal-engine/pcg-development-guides
36. 程序化内容生成概述| 虚幻引擎5.7 文档| Epic Developer Community https://dev.epicgames.com/documentation/zh-cn/unreal-engine/procedural-content-generation-overview
37. Microservices For Java Developers | PDF | Computing - Scribd https://www.scribd.com/document/535856589/Microservices-for-Java-Developers
38. suse-sles-15-sp4-hardened-byos-v20240809-hvm-ssd-x86_64 ... https://publiccloudimagechangeinfo.suse.com/amazon/suse-sles-15-sp4-hardened-byos-v20240809-hvm-ssd-x86_64/package_changelogs.html

39. AWS Certified Developer - Associate (DVA-C01) Cert Guide - Scribd https://www.scribd.com/document/714194295/AWS-Certified-Developer-Associate-DVA-C01-Cert-Guide
40. [PDF] Data Generation for JSON Schema - HAL theses https://theses.hal.science/tel-04888295v1/file/2024UPSLD037.pdf
41. Web of Things (WoT): Use Cases and Requirements - W3C https://www.w3.org/TR/wot-usecases/
42. (PDF) Survey on JSON Data Modelling - ResearchGate https://www.researchgate.net/publication/327325420_Survey_on_JSON_Data_Modelling
43. Harm categories in Azure AI Content Safety - Microsoft Learn https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/harm-categories
44. SAGE: A Generic Framework for LLM Safety Evaluation Warning https://arxiv.org/html/2504.19674v1
45. Azure OpenAI in Microsoft Foundry Models REST API reference https://learn.microsoft.com/en-us/azure/foundry/openai/reference
46. SpecEval: Evaluating Model Adherence to Behavior Specifications https://arxiv.org/html/2509.02464v1
47. What are Foundry Tools? - Foundry Tools | Microsoft Learn https://learn.microsoft.com/en-us/azure/ai-services/what-are-ai-services
48. Responsible artificial intelligence governance: A review and ... https://www.sciencedirect.com/science/article/pii/S0963868724000672
49. Global AI Governance: Where the Challenge is the Solution - arXiv https://arxiv.org/html/2503.04766v1
50. Research Report on Global AI Governance https://www.wicinternet.org/pdf/ResearchReportonGlobalAIGovernance.pdf
51. Ethical Frameworks for AI and Blockchain Co-Governance https://www.researchgate.net/publication/396889235_Ethical_Frameworks_for_AI_and_Blockchain_Co-Governance
52. AI governance: a systematic literature review | AI and Ethics https://link.springer.com/article/10.1007/s43681-024-00653-w
53. On the Co-Design of Scientific Experiments and Industrial Systems https://arxiv.org/html/2603.26613v1
54. [PDF] Sensor Spoofing Attacks via Supply Chain Implants in Satellite ... https://arxiv.org/pdf/2603.10388
55. [PDF] i Speculative Physics: the Ontology of Theory and Experiment ... - arXiv https://arxiv.org/pdf/1406.5636

56. [PDF] A Unified DCIM Framework for AI Data Centers - arXiv https://arxiv.org/pdf/2601.04750
57. [PDF] MI9: An Integrated Runtime Governance Framework for Agentic AI https://www.arxiv.org/pdf/2508.03858v3
58. A Full-Stack Benchmark for Privacy Leakage in Multi-Agent LLM ... https://arxiv.org/html/2602.11510v2
59. [PDF] Benchmarking Reasoning Reliability in Artificial Intelligence Models ... https://arxiv.org/pdf/2510.19836
60. A Governance-First Paradigm for Principled Agent Engineering - arXiv https://arxiv.org/html/2510.13857v1
61. A Full-Stack Benchmark for Privacy Leakage in Multi-Agent LLM ... https://arxiv.org/html/2602.11510v1
62. Reinforcement Learning Foundations for Deep Research Systems https://arxiv.org/html/2509.06733v1
63. Fundamentals and Practical Implications of Agentic AI - arXiv https://arxiv.org/html/2505.19443v1
64. [PDF] ThinkFL: Self-Refining Failure Localization for Microservice Systems ... https://arxiv.org/pdf/2504.18776
65. WIO: Upload-Enabled Computational Storage on CXL SSDs - arXiv https://arxiv.org/html/2604.02442v1
66. Defining AI Models and AI Systems: A Framework to Resolve the ... https://arxiv.org/html/2603.10023v1
67. Internal Deployment Gaps in AI Regulation - arXiv https://arxiv.org/html/2601.08005v2
68. Governance-as-a-Service: A Multi-Agent Framework for AI System ... https://arxiv.org/html/2508.18765v1
69. [PDF] AI Governance and the Evolution of Institutions - arXiv https://arxiv.org/pdf/2507.13616
70. Approaches to Responsible Governance of GenAI in Organizations https://arxiv.org/html/2504.17044v1
71. Ethics Practices in AI Development: An Empirical Study Across ... https://arxiv.org/html/2508.09219v2
72. Managing policies and features for GitHub Copilot in your organization https://docs.github.com/copilot/managing-copilot/managing-github-copilot-in-your-organization/managing-github-copilot-features-in-your-organization/managing-policies-for-copilot-in-your-organization

73. Managing policies and features for GitHub Copilot in your enterprise https://docs.github.com/enterprise-cloud@latest/copilot/managing-copilot/managing-copilot-for-your-enterprise/managing-policies-and-features-for-copilot-in-your-enterprise
74. [PDF] System Security Configuration Guide for Cisco ASR 9000 Series ... https://www.cisco.com/c/en/us/td/docs/routers/asr9000/software/26xx/system-security/configuration/guide/b-system-security-cg-asr9000-26xx.pdf
75. [PDF] Oracle PaaS and IaaS Universal Credits Service Descriptions https://www.oracle.com/anz/contracts/docs/paas_iaas_universal_credits_3940775.pdf?download=false
76. CLAG: Adaptive Memory Organization via Agent-Driven Clustering ... https://arxiv.org/html/2603.15421v1
77. A zero-shot LLM framework for multimodal grievance classification ... https://www.nature.com/articles/s41598-025-32079-7
78. [PDF] OpenPort Protocol: A Security Governance Specification for AI Agent ... https://arxiv.org/pdf/2602.20196
79. Machine-Readable Runtime Governance for Autonomous AI Agents https://www.researchgate.net/publication/397007270_Policy_Cards_Machine-Readable_Runtime_Governance_for_Autonomous_AI_Agents
80. llama-3.1-nemoguard-8b-content-safety Model by NVIDIA https://build.nvidia.com/nvidia/llama-3_1-nemoguard-8b-content-safety/modelcard
81. [PDF] SAGE: A Generic Framework for LLM Safety Evaluation https://aclanthology.org/2025.emnlp-industry.2.pdf
82. Stripe webhooks issue in dev environment https://stackoverflow.com/questions/76964588/stripe-webhooks-issue-in-dev-environment
83. [PDF] Spare Strategy Analysis and Design for Large-Scale Satellite ... - arXiv https://arxiv.org/pdf/2509.09957
84. Lightspeed Data Compute for the Space Era - arXiv https://arxiv.org/html/2601.17589v2
85. [PDF] Characterization of Mega-Constellation Links for LEO Missions With ... https://arxiv.org/pdf/2509.00766
86. First Multi-Constellation Observations of Navigation Satellite Signals ... https://arxiv.org/html/2601.06081v2
87. Advancing Free-Space Optical Communication System Architecture https://arxiv.org/html/2410.23470v2
88. [PDF] Constellation Shaping for Bit-Interleaved LDPC Coded APSK - arXiv https://arxiv.org/pdf/1207.2215

89. Chemically peculiar stars investigated by the BRITE Mission - arXiv https://arxiv.org/html/2601.19782v1
90. [PDF] Performance Limits for Signals of Opportunity-Based Navigation https://arxiv.org/pdf/2407.16426
91. [PDF] Moduli of G-constellations and crepant resolutions II - arXiv https://arxiv.org/pdf/2209.11901
92. Orbital Debris in Earth Orbit: Operations, Stability, Control ... - arXiv https://arxiv.org/html/2603.23552v1
93. [PDF] A Multi-Agent Framework for AI System Compliance and Policy ... https://arxiv.org/pdf/2508.18765
94. Runtime Governance for Agentic AI Systems - MI9 - arXiv https://arxiv.org/html/2508.03858v1
95. A Multi-Level Case Analysis of AI Deployment Across Federal, State ... https://arxiv.org/html/2602.08728v1
96. Sovereign AI: Rethinking Autonomy in the Age of Global ... - arXiv https://arxiv.org/html/2511.15734v1
97. AI-Programmable Wireless Connectivity: Challenges and Research ... https://arxiv.org/html/2603.29752v1
98. AI Agent Systems: Architectures, Applications, and Evaluation - arXiv https://arxiv.org/html/2601.01743v1
99. TRiSM for Agentic AI: A Review of Trust, Risk, and Security ... - arXiv https://arxiv.org/html/2506.04133v5
100. The Auton Agentic AI Framework A Declarative Architecture for ... https://arxiv.org/html/2602.23720v1
101. Getting code suggestions in your IDE with GitHub Copilot https://docs.github.com/en/enterprise-cloud@latest/copilot/how-tos/get-code-suggestions/get-ide-code-suggestions?platform=windows&tool=vscode
102. Vibe coding with GitHub Copilot - GitHub Enterprise Cloud Docs https://docs.github.com/en/enterprise-cloud@latest/copilot/tutorials/vibe-coding
103. AI 内容安全的透明度说明和用例- Azure https://learn.microsoft.com/zh-cn/azure/foundry/responsible-ai/content-safety/transparency-note
104. Azure 机器学习提示流中的Content safety（文本）工具 https://learn.microsoft.com/zh-cn/azure/machine-learning/prompt-flow/tools-reference/content-safety-text-tool?view=azureml-api-2
105. Multi-Agent Coordination across Diverse Applications: A Survey - arXiv https://arxiv.org/html/2502.14743v1

106. Atlantes: A system of GPS transformers for global-scale real-time ... https://arxiv.org/html/2504.19036v1

107. Space AI Leveraging Artificial Intelligence for Space to Improve Life ... https://arxiv.org/html/2512.22399v1

108. Architectural Backdoors in Deep Learning: A Survey of ... - arXiv https://arxiv.org/html/2507.12919v1

109. Generative Diffusion Models for Wireless Networks - arXiv https://arxiv.org/html/2507.16733v3

110. [PDF] A Cartography of Open Collaboration in Open Source AI - arXiv https://www.arxiv.org/pdf/2509.25397

111. [PDF] Do AI Companies Make Good on Voluntary Commitments to ... - arXiv https://arxiv.org/pdf/2508.08345

112. A Survey on Large Language Model-based Agents in Autonomous ... https://arxiv.org/html/2505.12786v2

113. Integrating Artificial Intelligence into Operating Systems - arXiv https://arxiv.org/html/2407.14567v2

114. What is Azure AI Content Safety? https://learn.microsoft.com/en-us/azure/ai-services/content-safety/overview

115. Content Safety in Foundry Control Plane https://azure.microsoft.com/en-us/products/ai-services/ai-content-safety

116. Azure AI Content Safety documentation https://learn.microsoft.com/en-us/azure/ai-services/content-safety/

117. Azure AI Content Safety | Microsoft Azure Blog https://azure.microsoft.com/en-us/blog/product/azure-ai-content-safety/

118. Azure AI Content Safety Frequently Asked Questions https://learn.microsoft.com/en-us/azure/ai-services/content-safety/faq

119. No Addons enabled and getting massive LUA Errors https://us.forums.blizzard.com/en/wow/t/no-addons-enabled-and-getting-massive-lua-errors/2237629

120. Consents and Preferences Schema Field Group https://experienceleague.adobe.com/en/docs/experience-platform/xdm/field-groups/profile/consents

121. JSON SCHEMA NAMING AND DESIGN RULES ... https://unece.org/sites/default/files/2023-11/API-TECH-SPEC_JSON_Schema_NDR_version1p0.pdf

122. Consent Management Components https://help.sap.com/docs/SAP_CUSTOMER_DATA_CLOUD/8b8d6fffe113457094a17701f63e3d6a/414d6d1270b21014bbc5a10ce4041860.html

123. A snapshot of the JSON schema used for consent and ... https://www.researchgate.net/figure/A-snapshot-of-the-JSON-schema-used-for-consent-and-the-consent-modules-creation-or_fig5_359700101

124. From Logic Monopoly to Social Contract: Separation of Power and ... https://arxiv.org/html/2603.25100v1

125. [PDF] Reinforcement Learning Foundations for Deep Research Systems https://arxiv.org/pdf/2509.06733?

126. [PDF] AI Benchmark Democratization and Carpentry - arXiv https://arxiv.org/pdf/2512.11588

127. A Scalable Framework for Multi-Policy AI Compliance Evaluation https://arxiv.org/html/2601.11702v2

128. A Control-Theoretic Architecture for Governing Socio-Technical AI https://arxiv.org/html/2512.16873v1

129. [PDF] A Framework for Responsible AI Systems: Building Societal Trust ... https://arxiv.org/pdf/2503.04739

130. [PDF] System Security Configuration Guide for Cisco NCS 540 Series ... https://www.cisco.com/c/en/us/td/docs/iosxr/ncs5xx/system-security/26xx/b-system-security-cg-26xx-ncs540.pdf

131. List of Agent Configuration Parameters - Broadcom TechDocs https://techdocs.broadcom.com/us/en/symantec-security-software/identity-security/siteminder/12-8/configuring/web-agent-configuration/list-of-agent-configuration-parameters.html

132. Manage OIDC Objects Using REST APIs - TechDocs https://techdocs.broadcom.com/us/en/symantec-security-software/identity-security/siteminder/12-8/configuring/use-siteminder-as-openid-connect-provider/manage-oidc-objects-using-rest-apis.html

133. [PDF] Standardized Disclosure Schema + Evidence Pack + Exit/Liquidity ... https://www.sec.gov/files/ctf-written-input-daniel-bruno-corvelo-costa-012826.pdf